

Ros Day01 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.09

목차

1. 과제 목적
2. 패키지 및 노드 구현
3. C++ Publisher 구현
4. C++ Subscriber 구현
5. Python Publisher 구현
6. Python Subscriber 구현
7. 실행결
8. rqt_graph 확인

1. 과제 목적

이번 과제에서는 ROS2 의 Topic 통신을 이용하여 Publisher 와 Subscriber 노드 사이에서 데이터를 송수신하는 프로그램을 구현하였다.

C++ 패키지와 Python 패키지를 각각 제작하고, 각 패키지에 Publisher 와 Subscriber 노드를 작성하였다. 메시지 형식은 Int32, String, Float32, Bool 을 사용하였다.

또한 같은 패키지 내부에서의 Topic 통신뿐만 아니라 C++ 패키지와 Python 패키지 사이에서도 Topic 통신이 가능하도록 구성하고, Subscriber 가 수신한 데이터를 터미널에 출력하여 통신 결과를 확인하였다.

2. 패키지 및 노드 구성

이번 과제에서는 다음두개의 패키지를 생성하였다.

parkdoyeon_ros_rclcpp_pkg
parkdoyeon_ros_rclpy_pkg

C++ 패키지:

```
parkdoyeon_ros_rclcpp_pkg
├── include
│   ├── parkdoyeon_ros_rclcpp_pkg
│   │   ├── publisher.hpp
│   │   └── subscriber.hpp
├── src
│   ├── publisher.cpp
│   └── subscriber.cpp
├── CMakeLists.txt
└── package.xml
```

python 패키지:

```
parkdoyeon_ros_rclpy_pkg
├── parkdoyeon_ros_rclpy_pkg
│   ├── __init__.py
│   ├── publisher.py
│   └── subscriber.py
├── resource
├── setup.cfg
├── setup.py
└── package.xml
```

3. C++ Publisher 구현

3.1 Publisher 생성

create_publisher()을 이용하여 총 4개의 Publisher를 생성하였다.

각 Publisher는 다음과 같은 메시지 형식과 토픽을 사용한다.

```
cpp_int_topic    → Int32
cpp_string_topic → String
cpp_float_topic  → Float32
cpp_bool_topic   → Bool
```

코드는 다음과 같다.

```
int_publisher_ =
    this->create_publisher<std_msgs::msg::Int32>(
        "cpp_int_topic", 10);

string_publisher_ =
    this->create_publisher<std_msgs::msg::String>(
        "cpp_string_topic", 10);

float_publisher_ =
    this->create_publisher<std_msgs::msg::Float32>(
        "cpp_float_topic", 10);

bool_publisher_ =
    this->create_publisher<std_msgs::msg::Bool>(
        "cpp_bool_topic", 10);
```

3.2 메시지 발행

timer_callback() 함수는 1초마다 호출되도록 설정하였다.

콜백 함수가 실행될 때 Int32, String, Float32, Bool 메시지 객체를 생성하고 각각 값을 저장한 뒤 publish()를 이용해 메시지를 발행하였다.

Bool 데이터는 count_ 값에 따라 true와 false가 번갈아 나오도록 구현하였다.

코드는 다음과 같다

```
void PublisherNode::timer_callback()
{
    std_msgs::msg::Int32 int_msg;
    std_msgs::msg::String string_msg;
    std_msgs::msg::Float32 float_msg;
    std_msgs::msg::Bool bool_msg;

    int_msg.data = count_;
    string_msg.data = "Hello from C++";
```

```

float_msg.data = 3.14f;
bool_msg.data = (count_ % 2 == 0);

int_publisher_>publish(int_msg);
string_publisher_>publish(string_msg);
float_publisher_>publish(float_msg);
bool_publisher_>publish(bool_msg);

count_++;
}

```

4. C++ Subscriber 구현

4.1 Topic 구독

create_subscription()을 이용하여 C++ Publisher 에서 발행되는 토픽과 Python Publisher 에서 발행되는 토픽을 모두 구독하도록 하였다.

cpp_int_topic 은 C++ 패키지 내부 통신을 확인하기 위해 사용하였고, py_int_topic 은 Python 패키지에서 발행된 메시지를 C++ Subscriber 가 수신하기 위해 사용하였다.

다른 String, Float32, Bool 메시지도 같은 방식으로 각각의 토픽을 구독하도록 구성하였다.'

전체 코드는 다음과 같다.

```

cpp_int_subscriber_ =
    this->create_subscription<std_msgs::msg::Int32>(
        "cpp_int_topic",
        10,
        std::bind(
            &SubscriberNode::cpp_int_callback,
            this,
            std::placeholders::_1));

py_int_subscriber_ =
    this->create_subscription<std_msgs::msg::Int32>(
        "py_int_topic",
        10,
        std::bind(
            &SubscriberNode::py_int_callback,
            this,
            std::placeholders::_1));

```

4.2 Callback 을 이용한 데이터 출력

```

void SubscriberNode::cpp_int_callback(
    const std_msgs::msg::Int32::SharedPtr msg)
{
    RCLCPP_INFO(
        this->get_logger(),
        "[C++ -> C++] int: %d",

```

```

    msg->data);
}

void SubscriberNode::py_int_callback(
    const std_msgs::msg::Int32::SharedPtr msg)
{
    RCLCPP_INFO(
        this->get_logger(),
        "[Python -> C++] int: %d",
        msg->data);
}

```

Subscriber 가 메시지를 수신하면 등록된 callback 함수가 실행된다.

수신된 데이터는 msg->data 를 통해 확인하고, RCLCPP_INFO()를 이용하여 터미널에 출력하도록 구현하였다.

출력 문자열에 [C++ -> C++], [Python -> C++]과 같이 통신 방향을 표시하여 어느 Publisher 에서 전달된 데이터인지 확인하기 쉽게 구성하였다.

5. Python Publisher 구현

5.1 Publisher 생성

```

self.int_publisher = self.create_publisher(
    Int32,
    'py_int_topic',
    10
)

self.string_publisher = self.create_publisher(
    String,
    'py_string_topic',
    10
)

self.float_publisher = self.create_publisher(
    Float32,
    'py_float_topic',
    10
)

self.bool_publisher = self.create_publisher(
    Bool,
    'py_bool_topic',
    10
)

```

Python 에서는 rclpy 를 이용하여 Publisher 노드를 구현하였다.

C++ Publisher 와 마찬가지로 Int32, String, Float32, Bool 네 가지 메시지 타입에 대해 각각 Publisher 를 생성하였다.

사용한 토픽은 다음과 같다.

```
py_int_topic    → Int32
py_string_topic → String
py_float_topic  → Float32
py_bool_topic   → Bool
```

5.2 메시지 발행

timer_callback() 함수가 1 초마다 실행되도록 설정하고, 각 메시지에 값을 저장한 뒤 publish()를 이용하여 토픽으로 발행하였다.

Int32 는 count 값을 증가시키며 전송하였고, String 은 "Hello from Python", Float32 는 2.71, Bool 은 count 값에 따라 True 와 False 가 반복되도록 구성하였다.

```
def timer_callback(self):
    int_msg = Int32()
    string_msg = String()
    float_msg = Float32()
    bool_msg = Bool()

    int_msg.data = self.count
    string_msg.data = 'Hello from Python'
    float_msg.data = 2.71
    bool_msg.data = self.count % 2 == 0

    self.int_publisher.publish(int_msg)
    self.string_publisher.publish(string_msg)
    self.float_publisher.publish(float_msg)
    self.bool_publisher.publish(bool_msg)

    self.count += 1
```

6. Python Subscriber 구현

6.1 Topic 구독

```
self.cpp_int_subscriber = self.create_subscription(
    Int32,
    'cpp_int_topic',
    self.cpp_int_callback,
    10
)

self.py_int_subscriber = self.create_subscription(
```

)

Python Subscriber 에서도 C++에서 발행한 토픽과 Python 에서 발행한 토픽을 모두 구독하도록 작성하였다.

cpp_int_topic 을 구독하여 C++ → Python 통신을 확인하고, py_int_topic 을 구독하여 Python 패키지 내부 통신을 확인하였다.

나머지 String, Float32, Bool 메시지도 같은 방식으로 구독하였다.

6.2 Callback 출력

```
def cpp_int_callback(self, msg):
    self.get_logger().info(
        f'[C++ -> Python] int: {msg.data}')
    )
```

```
def py_int_callback(self, msg):
    self.get_logger().info(
        f'[Python -> Python] int: {msg.data}')
    )
```

메시지를 수신하면 지정된 callback 함수가 실행되고, msg.data 를 이용하여 수신된 값을 확인하였다.

self.get_logger().info()를 이용해 수신된 데이터를 터미널에 출력하고, 통신 방향을 함께 출력하여 패키지 내부 통신과 패키지 간 통신을 구분할 수 있도록 하였다.

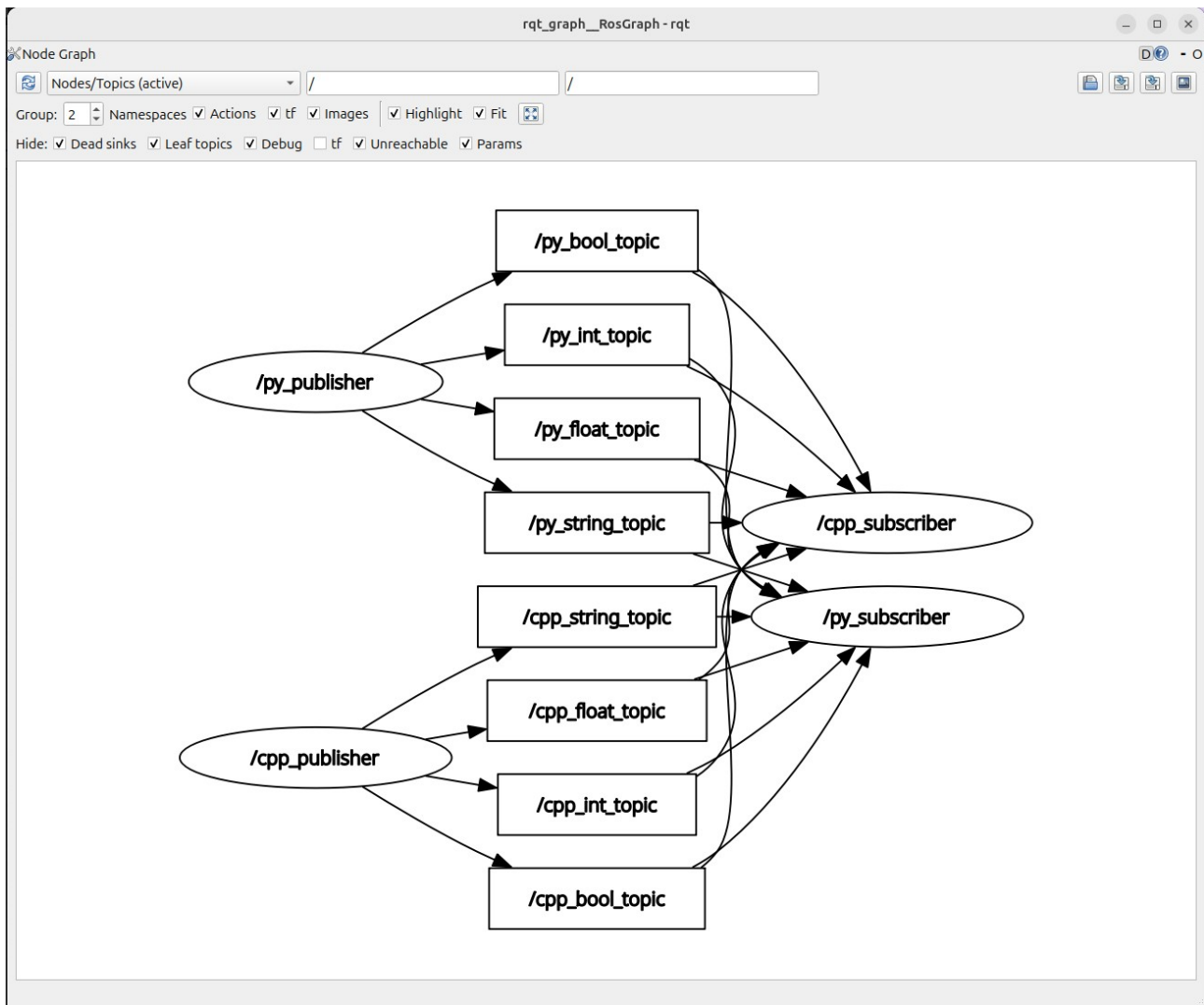
7. 실행 결과

7.1 C++ 패키지 내부 통신

[illegible]

[illegible]

8. rqt_graph 확인



rqt_graph 를 실행하여 노드와 토픽의 연결 구조를 확인하였다.

C++ Publisher 와 Python Publisher 에서 각각 Int32, String, Float32, Bool 타입의 토픽을 발행하고 있으며, C++ Subscriber 와 Python Subscriber 가 해당 토픽을 모두 구독하고 있는 구조를 확인할 수 있었다.

이를 통해 동일 패키지 내부 통신과 C++ 및 Python 패키지 간 통신 구조가 정상적으로 연결되어 있음을 알수 있었다.